

5.1 Memory Mapped I/O, I/O Mapped I/O

Address space partitioning in 8085

There are two techniques through which devices can be interfaced to microprocessor.

1. Memory mapped I/O
2. Peripheral mapped I/O or I/O mapped I/O

Memory mapped I/O

In memory mapped I/O scheme we can use only one address space. This particular one address space is allocated to both memory and I/O devices.

In total memory addresses, some addresses are assigned to memories and some to I/O devices. But we have to assign the address for I/O devices are different from the addresses which have been assigned to memories.

In this scheme remember that I/O device is also treated as a memory location.

Now take a very good example, MOV C, M instruction would transfer one byte of data from a memory location or it can also transfer an input device to the register C, depending on whether the address in the H-L register pair is assigned to a memory location or to an input device.

If H-L contains address of a memory location, data will be transferred from that memory location to register C, while if H-L pair contains the address of an input device, data will be transferred from that input device to register C.

I/O mapped I/O

In this method separate address space is given to I/O devices. Each I/O device is given an 8-bit address. Hence maximum 256 devices can be interfaced to the processor. The address range for the I/O devices is 00H-FFH. I/O control signals are used to perform read, write operations.

For reading data from I/O device or writing data to I/O device IN, OUT instructions needs to be used. We know that some CPUs provide one or more control lines like IO/M line for 8085, which indicates the status of operation, is memory or I/O operation.

If we get the status of IO/M line is high, it indicates I/O operation and when we get low, it points to memory operation. In this case the same address may be assigned to both memory and an I/O device depending on the status of IO/M line.

The above scheme is referred as I/O mapped I/O scheme. Look in this scheme two separate address spaces exist. One space is meant exclusively for memory operations and the other for I/O operations. The following figure shows, pictorially, both the schemes. Here it is assumed that the system has a 64 KB of memory and 256 I/O space.

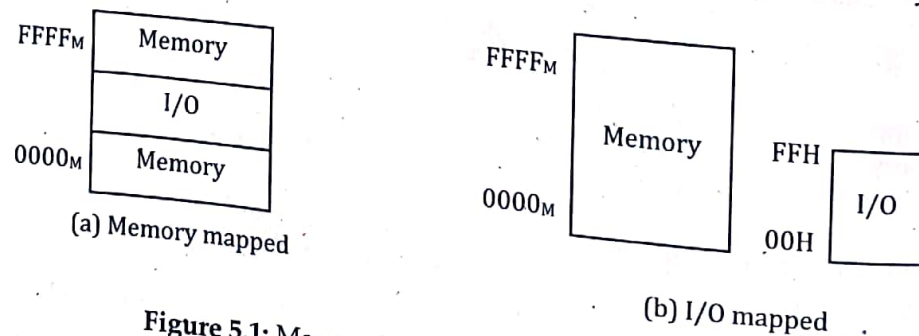


Figure 5.1: Memory map I/O and I/O mapped I/O.

Difference between Memory Mapped IO and IO Mapped IO

Definition

Memory mapped IO is a method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses one address space for memory and IO devices. IO mapped IO is a method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses two separate address spaces for memory and IO devices. Thus, this definition explains the basis of the difference between memory mapped IO and IO mapped IO.

Address Spaces

The main difference between memory mapped IO and IO mapped IO is that the memory mapped IO uses the same address space for both memory and IO devices. IO mapped IO uses two separate address spaces for memory and IO device.

Addresses for Memory

Branching from the above, there is another difference between memory mapped IO and IO mapped IO. As the memory mapped IO uses one address space for both IO and memory, the available addresses for memory are minimum due to the additional addresses for IO. In IO mapped IO, all the addresses can be used by the memory.

Instructions

While memory mapped IO uses the same instructions for both IO and memory operations, IO mapped IO uses separate instructions for read and write operations in IO and memory. We can say this as one other difference between memory mapped IO and IO mapped IO.

Efficiency

Moreover, memory mapped IO is less efficient while IO mapped IO is more efficient.

Conclusion

Memory mapped IO and IO mapped IO are two methods to perform input/output operations between the CPU and peripheral devices in the computer. The basic difference between memory mapped IO and IO mapped IO is that memory mapped IO uses the same address space for both memory and IO device while IO mapped IO uses two separate address spaces for memory and IO device.

Memory Mapped IO Versus IO Mapped IO

Memory Mapped IO	IO Mapped IO
A method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses one address space for memory and IO devices.	A method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses two separate address space for memory and IO device.
Uses the same address space for both memory and IO devices.	Uses the two separate address space for memory and IO device.
As the memory mapped IO uses one address space for both IO and memory, the available address for memory are minimum.	All the addresses can be used by the memory.
Uses the same instructions for both IO and memory operations	Uses separate instructions for read and write operations in IO and memory.
Less efficient	More efficient

Table 5.1: Difference between memory mapped IO and IO mapped IO.

eg: used in RAM/ROM, memory mapping is used.

In 8255A, 8257A I/O mapping is used.

5.2 Direct Memory Access (DMA)

The data transfer technique in which peripherals manage the memory buses for direct interaction with main memory without involving the CPU is called direct memory access (DMA). Using DMA technique large amounts of data can be transferred between memory and the peripheral without severely impacting CPU performance. During DMA transfer, the CPU is idle and has no control of the memory buses. A DMA controller takes over the buses to manage the transfer directly between the I/O device(s) and main memory. The structure of DMA controller is described below.

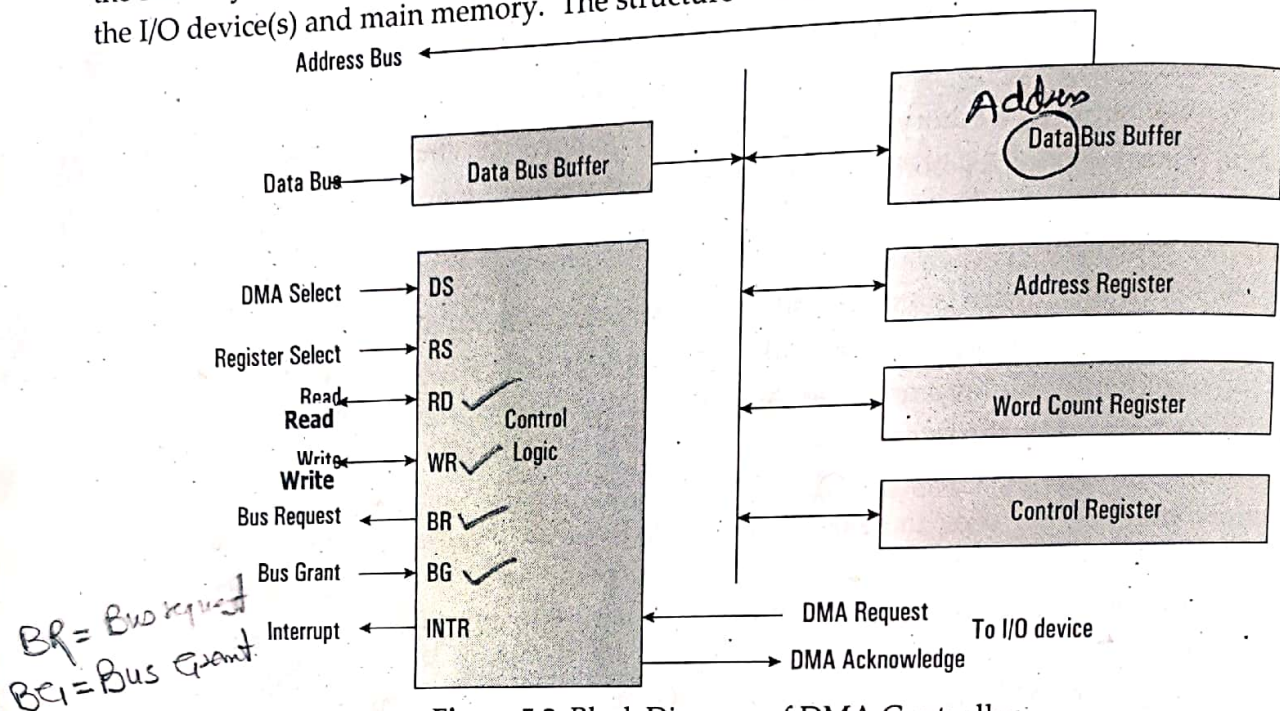


Figure 5.2: Block Diagram of DMA Controller.

The control unit communicates the CPU via data bus and control lines. The DMA controls/relinquishes the system bus using BR (Bus Request) and BG (Bus Grant) signals. DMA operates read and write operations via RD (Read) and WR (Write) signals. DMA sends request and acknowledge to I/O devices via DMA request and DMA acknowledge signals. The registers in DMA are selected by CPU through the address bus by enabling DS (DMA Select) and RS (Register Select) inputs. All registers in the DMA appear to the CPU as I/O interface registers. The address register contains an address to specify the desired location in memory. It is incremented after each word that is transferred to the memory. The word count register holds the number of words to be transferred. It is decremented by one after each word transfer and internally tested for zero. The control register specifies the mode of transfer. The DMA transfer operation is described below:

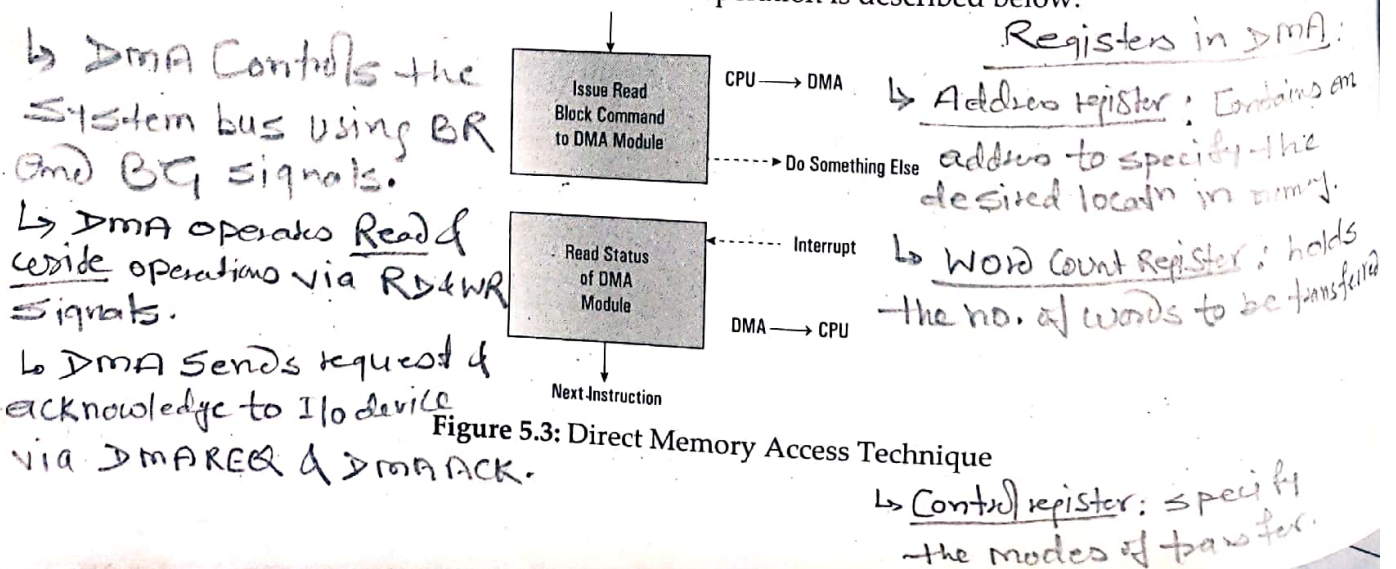


Figure 5.3: Direct Memory Access Technique

The DMA controller requests CPU to handle control of buses to the DMA using bus request (BR) signal. The CPU grants the control of buses to DMA using bus grant (BG) signal after placing the address bus, data bus and read and write lines into high impedance state (which behave like open circuit). CPU initializes the DMA by sending following information through the data bus.

1. Starting address of memory block for read or write operation.
2. The word count which is the number of words in the memory block.
3. Control to specify the mode of transfer such as read or write.
4. A control to start the DMA transfer.

The DMA takes control over the buses and directly interacts with memory and I/O units to transfer the data without CPU intervention. When the transfer completes, DMA disables the BR line. Thus CPU disables BG line, takes control over the buses and returns to its normal operation. The DMA transfer operation is illustrated below:

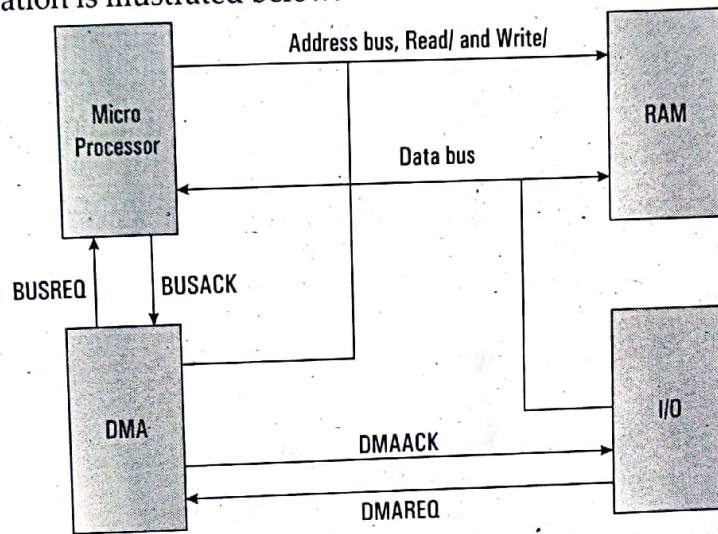


Fig 5.4: Illustration for DMA transfer in a computer system

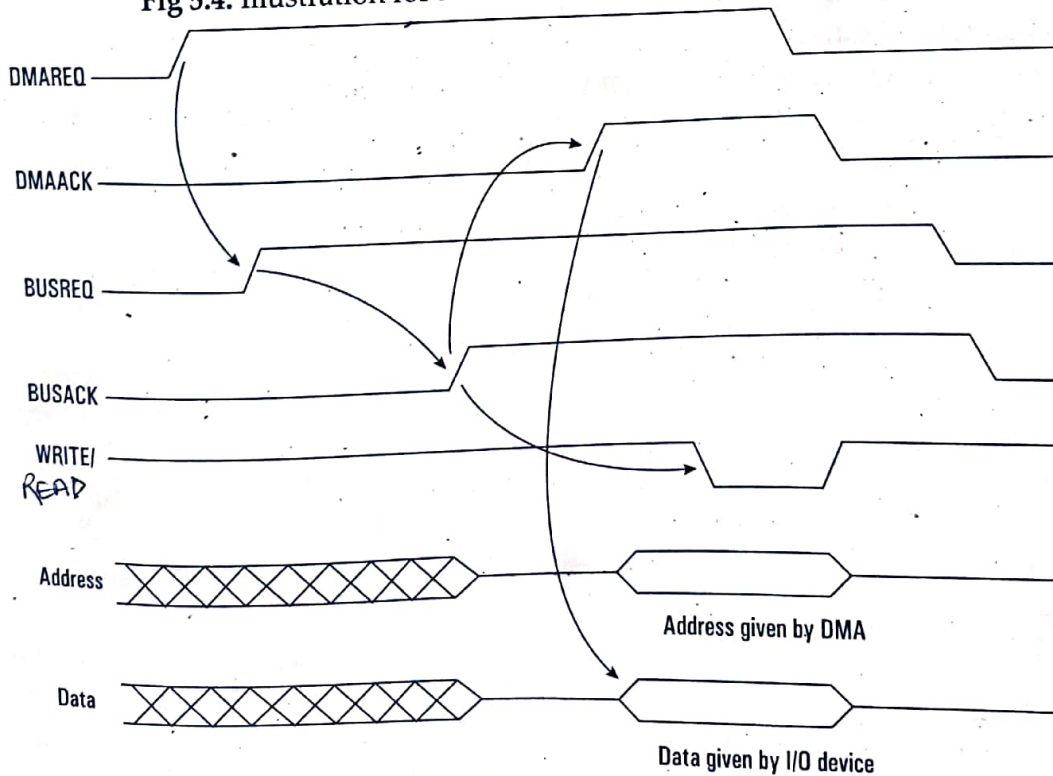


Figure 5.5: DMA Timing Diagram

DMA Transfer Modes

There are 3 different modes of DMA data transfer. They vary by how DMA controller determines when to transfer data (but actual data transfer process remains the same in all three cases).

(a) Burst Mode

Entire block of data is transferred in one continuous sequence.

Once the DMA controller is granted access to the system bus by CPU, it transfer all bytes of data in the data block before relinquishing control of system buses back to the CPU.

This mode is useful for loading programs or data files into memory, but it keeps CPU idle for relatively long period of time.

(b) Cycle Stealing Mode

DMA controller obtains access to system bus as in burst mode; transfers one byte of data and returns the control of the system bus to CPU. It continually issues requests using Bus Request (BR) signals, transferring one byte of data per request, until it has transferred its entire block of data. (steals one CPU cycle).

The data block is not transferred as quickly as in burst mode, but the CPU is not idled for long period of time as in burst mode.

(c) Transparent Mode

DMA controller only transfers data when CPU is performing operations that do not use system buses.

The main advantage of this mode is that CPU never stops executing its program.

The main disadvantages of this mode are

- * Hardware needed to determine whether the CPU is not using the system buses can be quite complex and relatively expensive.

- * Requires highest time to transfer a block of data as compared to above two modes.

To serve as a data transfer processor, DMA controller should have:

- * A data bus
- * An address bus
- * Read/Write control signals
- * Control signal to disable its role as a peripheral and to enable it as a processor

Advantages

1. Quick data transfer because a dedicated piece of hardware transfers data from one computer location to another and only one or two bus read/write cycles are required per piece of data transferred.
2. Minimizes latency in servicing a data acquisition device because the dedicated hardware responds more quickly than interrupts and transfer time is short.
3. Minimizes latency reduces the amount of temporary storage (memory) required on an I/O device.
4. Processor is not used for holding the data transfer activity and is available for other processing activity.
5. Also in systems where the processor primarily operates out of its cache, data transfer actually occurring in parallel, thus increasing overall system utilization.

Floppy disk, CDROM, External hard disk etc.

Application

1. Extensively used for computer-based data acquisition applications including streaming data to disk, real-time screen data display and continuous data acquisition applications.
2. DMA was used for floppy disk I/O in the original PC and for hard disk I/O in later versions.
3. PC-based DMA technology, along with high-speed bus technology, is driven by data storage, communications and graphics needs-all of which require the highest rates of data transfer between system memory and I/O devices.
4. Data acquisition applications have the same needs and therefore can take advantage of the technology developed for larger markets.

DMA transfer mode:

(1) Burst mode.

(2) Cycle stealing mode.

(3) Transparent mode.

8237 DMA Controller and interface.

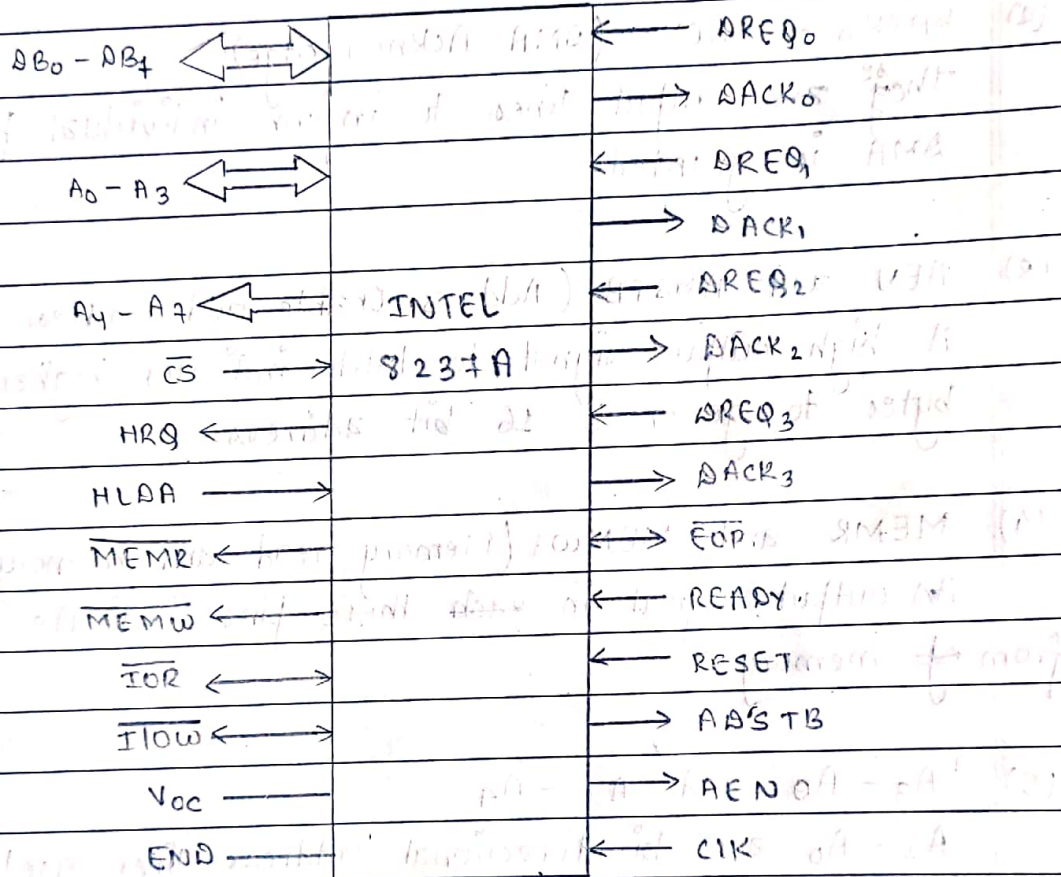


Fig: DMA controller and interface.

It is a high performance DMA. It is compatible with 8086, 8088 and 8085 microprocessor. It consists of 4 independent DMA channels, software ^{are} handle them. It has memory to memory transfer and automatic address increment and decrement. It has data transfer rate upto 1.6 MBps. It interface two types of devices i.e. MPU and peripherals (eg: floppy disk, pendrive, etc).

Some of the important DMA signals are:

- (1) $DREQ_0 - DREQ_3$ (DMA request)
They are integrated, asynchronous input signals to DMA channels from peripherals. A request is generated by activating the $DREQ$ line to obtain DMA service.
- (2) $DACK_0 - DACK_3$ (DMA Acknowledge)
They are output lines to inform individual peripheral that DMA is granted.
- (3) AEN and $A\overline{STB}$ (Address Enable and address strobe)
A high output signal to latch indicates higher order address bytes to generate 16 bit address.
- (4) $MEMR$ and $MEMW$ (Memory read and memory write)
An output signal in each these pins indicate read and write from ~~of~~ memory.
- (5) $A_3 - A_0$ and $A_7 - A_4$
 $A_3 - A_0$ are bi-directional address lines used as input to access control registers. During DMA cycle, these lines are used as output lines to generate a low order address that combines with the remaining address lines $A_7 - A_4$.
- (6) HRQ and $HLAA$ (Hold Request and Hold Acknowledge)
 HRQ is the output signal used to request the MPU for control of the system bus. After receiving the HRQ , the MPU completes the bus cycle in process and ~~for~~ issue the $HLAA$.

5.3 Interrupt

Interrupt is a mechanism by which an I/O or an instruction can suspend the normal execution of processor and get itself serviced. In response to an interrupt, the processor stops what it is currently doing and executes a service routine. When the execution of the service routine is terminated, the original process may resume its previous operation. The interrupt is initiated by an external device and is asynchronous, meaning that it can be initiated at any time without reference to system clock. However, the response to an interrupt request is directed or controlled by the microprocessor.

Interrupts are primarily issued on:

1. Initiation of I/O operation. ✓
2. Completion of I/O operation. ✓
3. Occurrence of hardware or software errors. ✓

Need for Interrupts

Interrupt is a signal sent by an external device to the processor, to the processor to perform a particular task or work. Mainly in the microprocessor based system the interrupts are used for data transfer between the peripheral and the microprocessor.

When a peripheral is ready for data transfer, it interrupts the processor by sending an appropriate signal to the interrupt pin of the processor. If the processor accepts the interrupt then the processor suspends its current activity and executes an interrupt service subroutine to complete the data transfer between the peripheral and the processor. After executing the interrupt service routine the processor resumes its current activity. This type of data transfer scheme is called interrupt driven data transfer scheme.

i) Interrupts are used for data transfer betⁿ peripherals & mp.

↳ **Interrupt driven data transfer scheme.**

Interrupt service routine resides inside a specified memory location.

Process of Interrupt Operation

↳ CPU saves PSW, PC (location of next instrn to be executed), into stack.

1. The I/O unit issues an interrupt signal to the processor. An interrupt signal from I/O is the request for exchange of data with the processor.
2. The processor finishes execution of the current instruction before responding to the interrupt.
3. The processor tests for an interrupt, determines that there is one, and sends an acknowledgement signal to the device that issued the interrupt. After receiving this acknowledgement, the device retains its interrupts signal.
4. The processor now begins to transfer the control to the routine which serves the interrupt request from the device. This routine is called 'Interrupt service routine' and it resides at a specified memory location.

For this process, the CPU needs to save information needed to reassume the current program at the point of interrupt. The minimum information required is (i) the status of the processor, which is contained by the process or status word (PSW) and (ii) the location of the next instruction to be executed which is contained by the program counter (PC), these all are pushed onto the stack.

5. The processor then loads the program counter with the entry location of the interrupt service routine that will respond to this interrupt. Once the program counter has been loaded, the control is transferred to the interrupt handler program. ↳ Interrupt handler program.
6. The fundamental requirement of the interrupt service routine is that it should begin by saving the contents of all the registers on the stack (as state of the main program should be safe). Suppose the user program is interrupted after the instruction at location N. The contents of all the registers plus the address of the next instruction are saved on the stack. The stack pointer is updated and the program counter is updated to point to the beginning of the interrupt service routine.
7. The interrupt handler now proceeds to process the interrupt. This will include an examination of status information relating to the I/O operation or the other event that caused an interrupt. It may also involve sending additional commands or acknowledgement to the I/O unit.
8. When interrupt processing is complete the saved register's value (of the main program) are retrieved from the stack and restored to the register.
9. The final function is to restore the PSW and program counter values from the stack. As a result the next instruction to be executed will be from the previously interrupted main program.

Types of Interrupts

There are three major types of interrupts that cause a break in the normal execution of a program. They can be classified as:

1. External interrupts
2. Internal interrupts
3. Software interrupts.

1. External Interrupts

↳ Initiated by external I/O devices.

External interrupts are initiated via the microprocessor's interrupt pins by external devices I/O devices, timing device, circuit monitoring the power supply, etc. Causes of these interrupts maybe; I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure. Time out interrupt may result from a program that is an endless loop and thus exceeded its time allocated. Power failure interrupt may have as its service routine a program that transfers the complete state of the CPU into a non-destructive memory in few milliseconds before power ceases.

An external device initiates the hardware interrupts of 8085 by placing an appropriate signal at the interrupt pin of the processor. The processor keeps on checking the interrupt pins at the second T-state of last machine cycle of every instruction. If the processor finds a valid interrupt signal and if the interrupt is unmasked and enabled, then the processor accepts the interrupt. The acceptance of the interrupt is acknowledged by sending an INTA signal to the interrupted device.

The processor saves the content of PC (program Counter) in stack and then loads the vector address of the interrupt in PC. (If the interrupt is non-vectored, then the interrupting device has to supply the address of ISR when it receives INTA signal). It starts executing ISR in this address. At the end of ISR, a return instruction, RET will be placed. When the processor executes the RET instruction, it POPs the content of the top of stack to PC. Thus the processor control returns to main program after servicing interrupt. The hardware interrupts of 8085 are TRAP, RST 7.5, RST 6.5, RST 5.5 and INTR.

External interrupts can be further divided into two types:

- * Maskable interrupt. ✓
- * Non-maskable interrupt. ✓

Maskable interrupt

A maskable interrupt is one which can be enabled or disabled by executing instructions such as EI (enable interrupts) and DI (Disable interrupt). If the microprocessor's interrupt enable flipflop is disabled, it ignores a maskable interrupt. In 8085, the 1 byte instruction EI sets the interrupt enable flipflop and enables the interrupt process. Similarly the 1 byte instruction DI resets the interrupt enable flipflop and disables the interrupt process. No maskable interrupts are recognized by the processor when the interrupt is disabled.

Non maskable interrupt

This type of interrupt cannot be enabled or disabled by instructions. This type has higher priority over the maskable interrupt. This means that if both the maskable and nonmaskable interrupts are activated at the same time, then the processor will service the non-maskable interrupt first. In 8085 TRAP is an example of nonmaskable interrupt.

2. Internal Interrupt *Register overflow, stack overflow, divide by zero*

Internal interrupts arise from illegal or erroneous use of an instruction or data. Cause of this interrupt may be: register overflow, attempt to divide by zero, an invalid operation code, stack overflow etc. These error conditions usually occur as a result of premature termination of the instruction execution. These are even termed as exceptions.

The difference between internal and external interrupt is that the internal interrupt is initiated by some exceptional conditions caused by the program itself rather than by an external event. Internal interrupts are synchronous with the program, while external interrupts are asynchronous. If the program is running, the internal interrupt will occur in the same place each time. External interrupts depend on external conditions that are independent of the program being executed at the time.

3. Software Interrupt : *is initiated by executing an instruction.*

External and internal interrupts are initiated from signal that occur in the hardware of the CPU. A software interrupt is initiated by executing an instruction. Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call. It can be used by the programmer to initiate an interrupt procedure at any desired point in the program. The most common use of software interrupt is associated with a supervisor call instruction. This instruction provides means for switching from a CPU user mode to the supervisor mode. Certain operations in the computer may be assigned to the supervisor mode only, as for example, a complex input or output transfer procedure. In 8085 the instruction like RST 0, RST 1, RST 2, RST 3, ..., etc., causes a software interrupt. The vector addresses of software interrupts are given in table below.

↳ Switching from a CPU User mode to the supervisor mode.

↳ Eg: RST 0, RST 1, RST 2, RST 3 are example of software interrupt.

Interrupt	Vector address
RST 0	0000H
RST 1	0008H
RST 2	0010H
RST 3	0018H
RST 4	0020H
RST 5	0028H
RST 6	0030H
RST 7	0038H

Interrupt	Vector address
RST 7.5	003CH
RST 6.5	0034H
RST 5.5	002CH
TRAP	0024H

Table 5.2: Vector address of software interrupts

The software interrupt instructions are included at the appropriate (or required) place in the main program. When the processor encounters the software instruction, it pushes the content of PC (Program Counter) to stack. Then loads the Vector address in PC and starts executing the Interrupt Service Routine (ISR) stored in this vector address. At the end of ISR, a return instruction - RET will be placed. When the RET instruction is executed, the processor POP the content of stack to PC. Hence the processor control returns to the main program after servicing the interrupt. Execution of ISR is referred to as servicing of interrupt.

All software interrupts of 8085 are vectored interrupts. The software interrupts cannot be masked and they cannot be disabled. The software interrupts are RST 0, RST 1, ... RST 7 (8 Nos).

Vectored and Non-Vectored Interrupts -

A vectored interrupt is where the CPU actually knows the address of the Interrupt Service Routine in advance. All it needs is that the interrupting device sends its unique vector via a data bus and through its I/O interface to the CPU. The CPU takes this vector, checks an interrupt table in memory, and then carries out the correct ISR for that device. So the vectored interrupt allows the CPU to be able to know what ISR to carry out in software (memory).

Vectored Interrupts are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address. Vector Addresses are calculated by the formula $8 * \text{TYPE}$

Interrupt	Vector address
TRAP (RST 4.5)	24 H
RST 5.5	2C H
RST 6.5	34 H
RST 7.5	3C H

$$8 \times 4.5 = 360d = 90H$$

For Software interrupts vector addresses are given by:

Interrupt	Vector address
RST 0	00 H
RST 1	08 H
RST 2	10 H
RST 3	18 H
RST 4	20 H
RST 5	28 H
RST 6	30 H
RST 7	38 H

INTR is the only non-vectored interrupt in 8085.

Non-Vectored Interrupts are those in which vector address is not predefined. The interrupting device gives the address of sub-routine for these interrupts. INTR is the only non-vectored interrupt in 8085 microprocessor.

A non-vectored interrupt is where the interrupting device never sends an interrupt vector. An interrupt is received by the CPU, and it jumps the program counter to a fixed address in hardware. This is literally a hard coded ISR which is device agnostic. MIPS uses this via its syscall instruction, which is the same instruction you use regardless what device, if it is an external event notifier (e.g. keyboard input), a completion signal (e.g. print completion), a clock interrupt to tell the CPU to allocate control to a different device, or an abnormal event indicator (e.g. power failure). The CPU crucially does not know which device caused the interrupt without polling each I/O interface in a loop and checking the status register of each I/O interface to find the one with status "interrupt created".

Maskable and Non-Maskable Interrupts-Maskable

Interrupts are those which can be disabled or ignored by the microprocessor. These interrupts are either edge-triggered or level-triggered, so they can be disabled. INTR, RST 7.5, RST 6.5, RST 5.5 are maskable interrupts in 8085 microprocessor.

Non-Maskable Interrupts are those which cannot be disabled or ignored by microprocessor. TRAP is a non-maskable interrupt. It consists of both level as well as edge triggering and is used in critical power failure conditions.

Interrupt Driven Data Transfer Scheme

The interrupt driven data transfer scheme is the best method of data transfer for effectively utilizing the processor time. In this scheme, the processor first initiates the I/O device for data transfer. After initiating the device, the processor will continue the execution of instructions in the program. Also at the end of an instruction the processor will check for a valid interrupt signal. If there is no interrupt then the processor will continue the execution.

When the I/O device is ready, it will interrupt the processor. On receiving an interrupt signal, the processor will complete the current instruction execution and saves the processor status in stack. Then the processor calls an interrupt service routine (ISR) to service the interrupted device. At the end of ISR the processor status is retrieved from stack and the processor starts executing its main program. The sequence of operations for an interrupt driven data transfer scheme is shown in figure below.

	Priority	
TRAP	1 (Highest)	0024H
RST 7.5	2	003CH
RST 6.5	3	0034H
RST 5.5	4	0026H
INTR	5	XXXX

Polled interrupt.

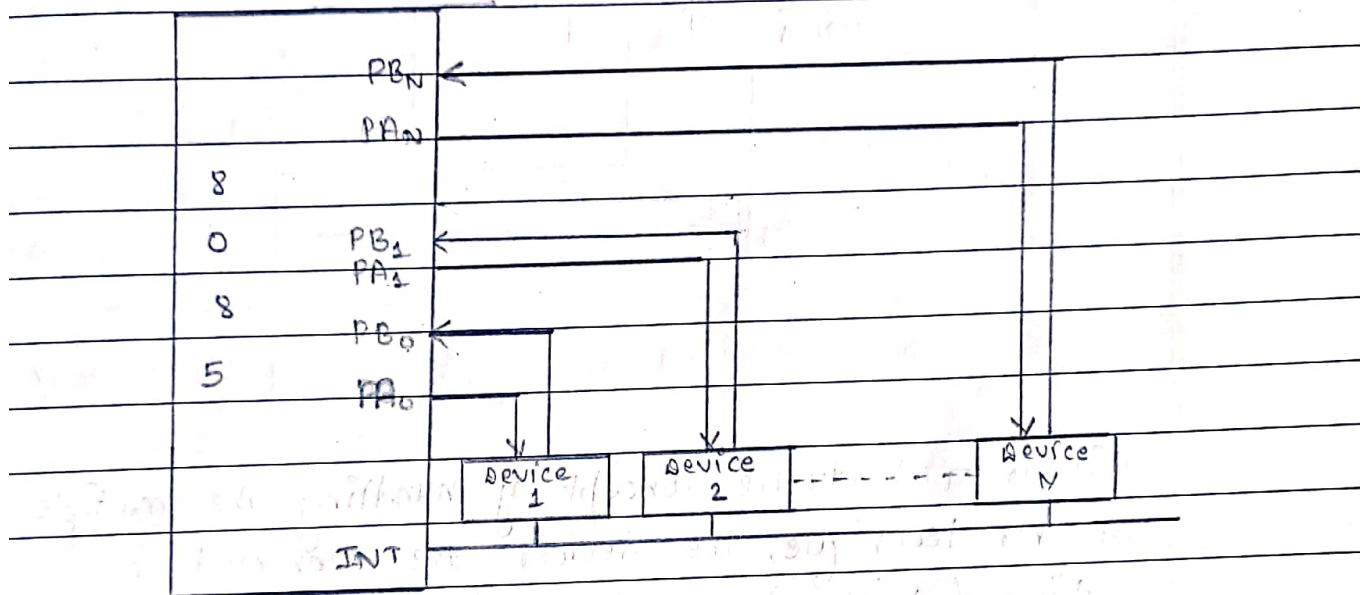


Fig: Polled interrupt.

Polled interrupts are handled using software and are therefore slower compared to vector (hardware) interrupt. In this method, there is one common branch attached for all interrupts. The program that takes care of interrupt begins at the branch address and polls (test) the interrupt sources in sequence. The order in which they are tested determines the parity of each interrupt. The highest parity source is tested first, and if its interrupt signal is on control branches to a service routine for this source. Otherwise, the next lower parity source is tested and so on.

Polled interrupts are very simple but on large number of devices, the time require to poll each device may exceed the service to the device. In such case, a faster mechanism called chained interrupt is used.

Chained (vectored) interrupt :

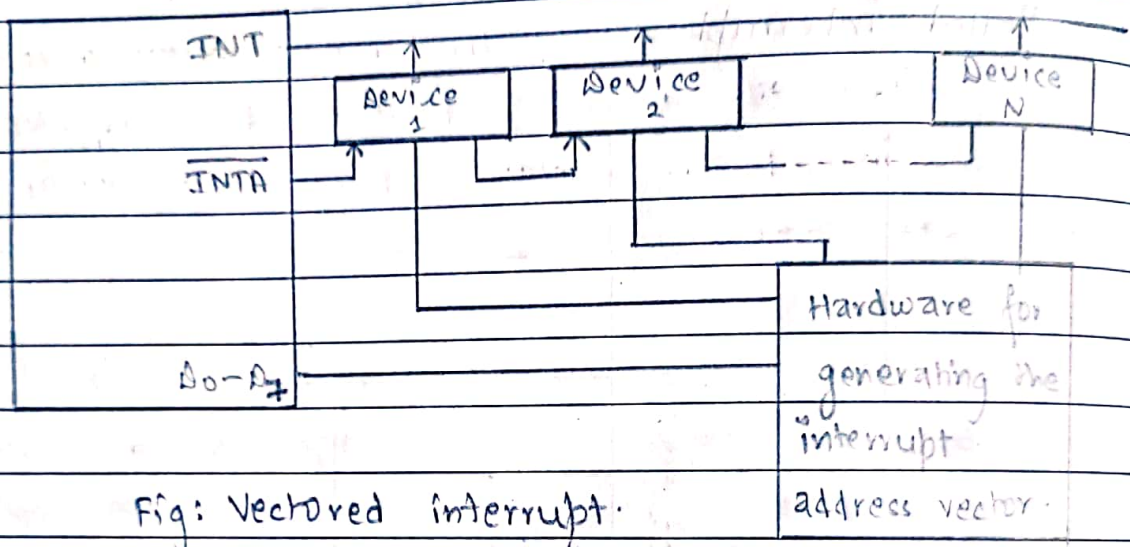


Fig: Vectored interrupt.

This is a hardware concept of handling the multiple interrupt. In this technique, the devices are connected in a chain fashion (structure) as shown in figure above/below.

Here, the device with highest ^{priority} priority is placed in the first position followed by lower priority devices. Suppose that, one or more devices interrupt the processor at a time. In response, the processor saves its current status and then generates an interrupt acknowledge (INTA) signal to the highest priority device. If this device has generated the interrupt, it will accept the INTA signal ~~except~~ ^{accept} the ~~INT~~ from the processor otherwise it will pass INTA on to the next device until the INTA is accepted by the interrupting device. With the help of hardware, it generates interrupt vector address.

5.4 The 8259 Programmable Interrupt Controller

The 8259A programmable interrupt controller designed to work with Intel microprocessors 8085, 8086 and 8088. The 8259A interrupt controller can

1. Manage eight interrupts according to the instructions written into its control registers. This is equivalent to placing eight interrupt pins on the processor in place of one INTR (8085) pin.
2. Vector can interrupt request anywhere in the memory map. However, all eight interrupts are spaced at the interval of either four or eight locations. This eliminates all the major drawback of the 8085 interrupts in which all interrupts are vectored to memory locations on page 00H.
3. Resolve eight levels of interrupt priorities in a variety of modes, such as fully nested mode, automatic rotation mode, and specific rotation mode.
4. Mask each interrupt request individually.
5. Read the status of pending interrupts, in-service interrupts, and masked interrupts.
6. Be set up to accept either the level-triggered or the edge-triggered interrupt request
7. Be expanded to 64 priority levels by cascading additional 8259As.
8. Be set up to work with either the 8085 microprocessor mode or the 886/8088 microprocessor mode.

The 8259A is upward-compatible with its predecessor, the 8259. The main difference between the two is that the 8259A can be used with Intel's 8086/88 16-bit microprocessor. It also includes additional features such as the level-triggered mode, buffered mode, and automatic-end-of interrupt mode. To simplify the explanation of the 8259A, illustrative examples will not include the cascade mode or the 8086/88 mode and will be limited to modes continuously used with the 8085.

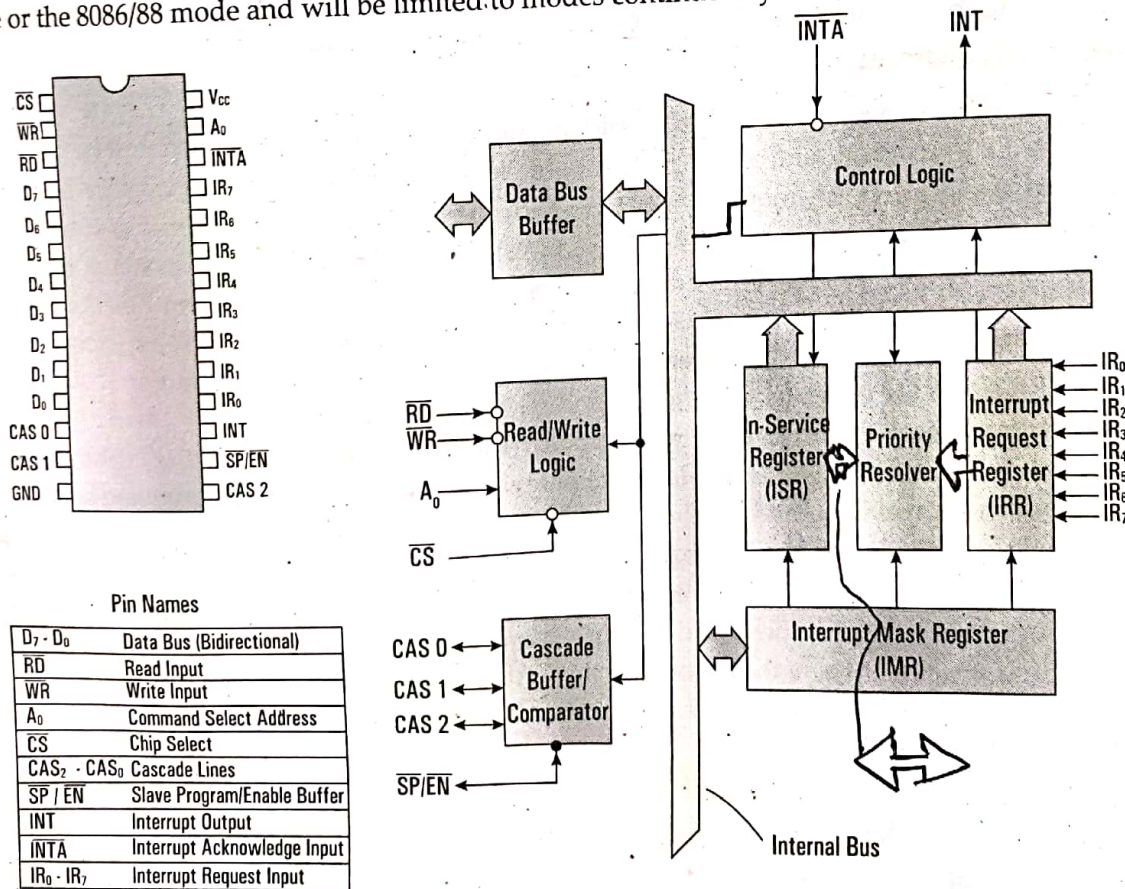


Figure 5.12: Block diagram of 8259A programmable interrupt controller

$\overline{SP/EN}$: slave program / Enable Buffer

The figure shows the internal block diagram of the 8259A. It includes eight blocks: control logic, Read/Write logic, data bus buffer, three registers (IRR, ISR and IMR), priority resolver, and cascade buffer. This diagram shows all the elements of a programmable device, plus additional blocks. The functions of some of these blocks need explanation, which is given below:

READ/WRITE LOGIC

This is a typical Read/Write control logic. When the address line A_0 is at logic 0, the controller is selected to write a command or read a status. The Chip Select logic and A_0 determine the port address of the controller.

CONTROL LOGIC

This block has two pins: INT (Interrupt) as an output, and INTA (Interrupt Acknowledge) as an input. The INT is connected to the interrupt pin of the MPU. Whenever a valid interrupt is asserted, this signal goes high. The INTA is the Interrupt Acknowledge signal from the MPU.

INTERRUPT REGISTERS AND PRIORITY RESOLVER

The interrupt Request Register (IRR) has eight input lines (IR_0 - IR_7) for the interrupts. When these lines go high, the requests are stored in the register. The In-Service Register (ISR) stores all the levels that are currently being serviced, and the Interrupt Mask Register (IMR) stores the masking bits of the interrupt lines to be masked. The Priority Resolver (PR) examines these three registers and determines whether INT should be sent to the MPU.

CASCADE BUFFER/COMPARATOR

This block is used to expand the number of interrupt levels by cascading two or more 8259As.

Interrupt Operation

To implement interrupts, the Interrupt Enable flip-flop in the microprocessor should be enabled by writing the EI instruction, and the 8259A should be initialized by writing control words in the control register. The 8259A requires two types of control words: Initialization Command Words (ICWs) and Operational Command Words (OCWs). The ICWs are used to set up the proper conditions and specify RST vector addresses. The OCWs are used to perform functions such as masking interrupts, setting up status-read operations, etc. After the 8259A is initialized, the following sequence of events occur when one or more interrupt request lines go high:

1. The IRR stores the requests.
2. The priority resolver checks three registers: the IRR for interrupt requests, the IMR for masking bits, and the ISR for the interrupt request being served. It resolves the priority and sets the INT high when appropriate.
3. The MPU acknowledges the interrupt by sending INTA.
4. After the INTA is received, the appropriate priority bit in the ISR is set to indicate which interrupt level is being served, and the corresponding bit in the IRR is reset to indicate that the request is accepted. Then, the opcode for the CALL instruction is placed on the data bus.
5. When the MPU decodes the CALL instruction, it places two or more INTA signals on the data bus.
6. When 8259A receives the second INTA, it places the low-order byte of the CALL address on the data bus. At the third INTA, it places the high-order byte on the data bus. The CALL address is the vector memory location for the interrupt: this address is placed in the control register during the initialization.

During the third INTA pulse, the ISR bit is reset either automatically (Automatic-End-of-Interrupt—AEOI) or by a command word that must be issued at the end of the service routine (End-of-Interrupt—EOI). This option is determined by the initialization command word (ICW). The program sequence is transferred to the memory location specified by the CALL instruction.

Priority Modes and Other Features

Many types of priority modes are available under software control in the 8259A, and they can be changed dynamically during the program by writing appropriate command words. Commonly used priority modes are discussed below:

1. **Fully Nested Mode:** This is a general-purpose mode in which all IRS (Interrupt Requests) are arranged from highest to lowest, with IR_0 as the highest and IR_7 as the lowest. In addition, any IR can be assigned the highest priority in this mode; the priority sequence will then begin at that IR. In the example below, IR_4 has the highest priority, and IR_3 has the lowest priority:

IR_0	IR_1	IR_2	IR_3	IR_4	IR_2	IR_3	IR_4
4	5	6	7	0	1	2	3

2. **Automatic Rotation Mode:** In this mode, a device, after being serviced, receives the lowest priority. Assuming that the IR_2 has just been serviced, it will receive the seventh priority, as shown below:

IR_0	IR_1	IR_2	IR_3	IR_4	IR_2	IR_3	IR_4
1	6	7	0	1	2	3	4

3. **Specific Rotation Mode:** This mode is similar to the automatic rotation mode, except that the user can select any IR for the lowest priority, thus fixing all other priorities.

End of Interrupt

After the completion of an interrupt service, the corresponding ISR bit needs to be reset to update the information in the ISR. This is called the End-of-Interrupt (EOI) command. It can be issued in three formats.

1. **Nonspecific EOI Command:** When this command is sent to 8259A, it resets the highest priority ISR bit.
2. **Specific EOI Command:** This command specifies which ISR bit to reset.
3. **Automatic EOI:** In this mode, no command is necessary. During the third INTA, the ISR bit is reset. The major drawback with this mode is that the ISR does not have information on which IR is being serviced. Thus, any IR can interrupt the service routine irrespective of its priority, if the Interrupt Enable flip-flop is set.

Additional Features of the 8259A

The 8259A is a complex device with various modes of operation. These modes are listed below (for reference; the user should refer to the 8085 User's manual for details.)

- ⌘ **Interrupt Triggering:** The 8259A can accept an interrupt request with either the edge triggered mode or the level-triggered mode. This mode is determined by the initialization instructions.
- ⌘ **Interrupt Status:** The status of the three instruction registers (IRR, ISR and IMR) can be read, and this status information can be used to make the interrupt process versatile.
- ⌘ **Poll Method:** The 8259A can be set up to function in a polled environment. The MPU polls the 8259A rather than each peripheral.